

StackSense

Architecture & Design Document

Push-based infrastructure monitoring platform

Version 1.0 - June 2026

Django 5.2 - PostgreSQL 15 - Redis - Docker

Confidential - internal documentation

Contents

- > 1. Introduction & Purpose
- > 2. Technology Stack
- > 3. Architecture Overview
- > 4. Data & Request Flow
- > 5. Component Breakdown
- > 6. Detection Subsystems
- > 7. Alerting & Notifications
- > 8. Data Model
- > 9. Security Model
- > 10. The Agent
- > 11. Deployment & Operations
- > 12. Configuration & Retention
- > 13. Current State & Recent Enhancements
- > Appendix A: Ingestion & Key API Endpoints
- > Appendix B: Scheduled Jobs
- > Appendix C: Roles & Capabilities

1. Introduction & Purpose

StackSense is a self-hosted infrastructure monitoring platform. Lightweight agents installed on monitored Linux servers collect system metrics and push them over HTTPS to a central Django application, which stores the data, analyzes it (anomalies, memory leaks, security events, uptime), raises role-routed alerts (email / Slack), and presents operations and executive dashboards.

The defining architectural choice is the PUSH model: agents only dial OUT to the StackSense server with a per-server bearer token. The server never opens a connection back to a monitored host and the agent never listens on a port. This keeps the monitored fleet's attack surface unchanged - no inbound rule, no credentials stored centrally for the hosts, and the agent runs unprivileged and read-only.

Design principles

- > Push, not poll - agents initiate; the server is never a client of the hosts.
- > Deny-by-default security - RBAC resolved server-side; per-server hashed tokens.
- > Transparent analytics - deterministic, explainable detection over black-box ML.
- > Operate simply - one Docker host, a one-command installer, and a safe updater.

2. Technology Stack

Layer	Technology	Notes
Language / runtime	Python 3.11	Server and agent
Web framework	Django 5.2	Project: log_analyzer
App server	Gunicorn	Behind nginx
Reverse proxy / TLS	nginx + certbot	Let's Encrypt or uploaded cert; prod overlay
Database	PostgreSQL 15	psycpg2; time-series + config
Cache / scheduler state	Redis 7	django-redis; app heartbeat, caches
Static files	WhiteNoise	CompressedManifestStaticFilesStorage
Frontend	Django templates + JS	Chart.js graphs; jazzmin admin theme
Analytics / ML libs	numpy, pandas, scikit-learn, adtk	Live detector is baseline stats, not ML
Optional LLM	Ollama	Disabled in production (LLM_ENABLED=False)
Agent	Python + psutil + stdlib	Runs as a hardened systemd service
Packaging / deploy	Docker Compose	base compose + prod overlay; setup.sh / update.sh
Auth	Django sessions + RBAC	Per-agent bearer tokens (SHA-256 hashed)

3. Architecture Overview

StackSense has three tiers: (1) the agents on monitored VMs, (2) the central server (nginx -> Unicorn -> Django, plus Postgres and Redis), and (3) a background scheduler process that runs the periodic analysis and maintenance jobs. All three run from the same Docker Compose stack on a single host; the agents run independently on each monitored machine.

Logical flow

Stage	What happens
1. Collect	Agent samples CPU/mem/disk/network, services, containers, SSH auth, top processes.
2. Push	Agent POSTs JSON to /api/agent/* over HTTPS with its bearer token (every ~30s).
3. Authenticate	Server verifies the token (hashed), binds the data to that one server, stamps server-side time.
4. Store	Rows written to Postgres (SystemMetric, Service, Container, SSHAuthEvent) + heartbeat.
5. Analyze	Scheduler jobs detect anomalies, leaks, security events; run uptime checks; track connectivity.
6. Alert	Threshold/availability breaches -> AlertHistory -> role-routed email/Slack. Anomalies -> dashboard bell.
7. Present	Operations/Executive dashboards, server detail, alerts, security, KPIs - server-rendered + Chart.js.

The web request path is: client -> nginx (terminates TLS on 443, or a forwarded port such as 1443) -> Unicorn -> Django. RBAC middleware authorizes every request (deny-by-default) before the view runs. The agent ingest endpoints use their own bearer-token auth and bypass the session RBAC.

4. Data & Request Flow

Two independent entry paths exist, with different authentication:

- > Machine path (agents): token-authenticated POSTs to /api/agent/metrics, /services, /containers, /ssh-auth, and /heartbeat. The token is the server's identity - a client cannot spoof which server it is by changing the JSON body.
- > Human path (operators): session-authenticated browser traffic to the dashboards and config pages, authorized by the RBAC capability map.

Ingested metrics are timestamped server-side (agents cannot back- or forward-date data). Every accepted push refreshes the server's heartbeat, which drives the online/offline/warning status. Raw metrics are rolled up into hourly/daily aggregates and pruned per the configured retention window.

5. Component Breakdown

5.1 Ingestion API (core/agent_api.py)

Token-authenticated endpoints that receive agent pushes. Responsibilities: verify the bearer token against the hashed AgentCredential, enforce a body-size limit, validate/whitelist fields, store rows, update the heartbeat, and honor per-server monitoring suspension. Service status changes feed availability alerts.

5.2 Background scheduler (metrics_scheduler.py)

A long-running loop that periodically invokes Django management commands: anomaly detection, memory-leak detection, security-event detection, synthetic (uptime) checks, connectivity checks, metric aggregation, retention pruning, and an app-heartbeat tracker (used to grant servers a grace period after the app restarts).

5.3 Web application & UI (core/views.py, templates)

Server-rendered Django templates with vanilla JS and Chart.js. Surfaces: the Operations and Executive dashboards, per-server detail (windowed trend charts, services, containers, processes), the servers list, alerts, the anomalies notification bell, security dashboard, business KPIs, synthetic checks, and user/role administration.

5.4 RBAC middleware (core/permissions.py)

The single source of truth for capabilities, the role->capability matrix, and the route->capability map. Enforced by middleware on every request; the UI reads the same capabilities to show or disable controls consistently.

6. Detection Subsystems

Subsystem	How it works
Anomaly detection	Per-metric (CPU/mem/disk): fires on a hard ceiling OR an upward deviation from the server's own recent baseline (robust median + MAD). Explanations name the heaviest process at that sample. Sustained-gate + absolute floors avoid noise. Transparent, no LLM.
Memory-leak detection	Linear-trend (slope + R-squared) over a window for system memory, per-process RSS, and SysV/IPC objects - flags sustained growth, not normal sawtooth. Names the culprit process.
Security events	Parses the SSH auth log (auth.log / secure) for failed logins and detects brute-force patterns -> SecurityEvent.
Synthetic / uptime	Scheduled TCP/HTTP checks against configured targets, with latency and status recorded.
Connectivity	Detects servers going down / recovering from heartbeat freshness; raises/clears connection alerts.
Server status	online / warning / offline derived from heartbeat age (tolerant 180s threshold) + active alerts; suspended servers read offline. Anomalies do NOT affect status.

7. Alerting & Notifications

Alerts and anomalies are deliberately separated. ALERTS are actionable health events (threshold breaches, a server unreachable, a monitored service/container down). ANOMALIES are statistical notifications surfaced only on a dashboard bell icon - they never appear on the alerts page and never change a server's health status.

Categories & severity

Every alert carries one of five categories - Resource/Performance, Availability, Security, Capacity & Health, Business - and a severity (Low/Medium/High/Critical). Rising-edge detection ensures a sustained breach alerts once per episode rather than every cycle.

Role-based routing & channels

An AlertRoutingRule matrix maps (role x category x minimum severity) to recipients drawn from user accounts, so (for example) the CEO is not paged for routine operational noise. Delivery channels are email (SMTP) and Slack (incoming webhook), both with test actions in the alert configuration UI.

8. Data Model (key entities)

Entity	Purpose
Server	A monitored host (name, IP, status, suspend flags).
AgentCredential	Per-server bearer token, stored only as a SHA-256 hash.
SystemMetric	One metrics sample: CPU/mem/disk/network + disk_usage + top_processes (JSON).
Service / Container	Agent-detected services (systemd/port, with banner-identified product) and containers.
Anomaly	A detected statistical anomaly (metric, value, severity, explanation, resolved).
AlertHistory	A raised alert (type, severity, status triggered/resolved, message, recipients).
SecurityEvent / SSHAuthEvent	Security findings and raw SSH auth events.
SyntheticCheck / Result	Uptime check definitions and their results.
BusinessKPI / Value	Business metrics pushed via a separate token.
MonitoringConfig	Per-server thresholds, monitored disks/services, anomaly sensitivity, suspension.
AppConfig	Singleton app settings (base_url, setup_completed, retention, timezone).
UserACL / Role / Privilege	RBAC: a user's role and the role's capabilities. AlertRoutingRule for routing.
ServerHeartbeat	Last contact time per server - drives online/offline status.

9. Security Model

- > Push-only - the server never connects to monitored hosts; the agent never listens. No inbound port is opened on monitored machines for monitoring.
- > Per-server tokens - each agent authenticates with a bearer token stored only as a SHA-256 hash; the token (not the payload) is the server's identity, and it can be revoked/regenerated.
- > Deny-by-default RBAC - capabilities are resolved server-side from the verified session user; a client can never supply its role. Every route maps to a required capability.
- > Roles - Admin (all), CEO (operational/business, no user/role admin or impersonation), Operator (view operations, read-only). Custom roles supported. Impersonation is audited and only allows stepping DOWN in privilege.
- > Hardened agent unit - runs as a dedicated non-root user with NoNewPrivileges, ProtectSystem=strict, ProtectHome, PrivateTmp.
- > Input hardening on ingest - body-size cap, JSON validation, field whitelist, server-stamped timestamps; secrets live in a gitignored .env.

10. The Agent (agent/stacksense_agent.py)

A single standalone Python script using psutil and the standard library only - no Django, no heavy dependencies. It installs into its own virtualenv and runs as a systemd service. Per cycle it collects metrics, services, containers, SSH auth events, and the top processes, then pushes them with retry/back-off.

- > Service identification is privilege-free: systemd units plus listening ports, with a 1-second loopback banner grab to name the real product (nginx vs Apache vs LiteSpeed, Exim, Dovecot, MySQL, ...), falling back to the protocol when a banner is hidden.
- > Works on modern systemd Linux distributions with Python 3 (Ubuntu/Debian, RHEL/Alma/Rocky/CloudLinux incl. cPanel, Fedora, Amazon Linux). Needs systemd + Python 3.
- > Resilient: if the server is briefly unreachable it retries and drops what it cannot send (it never spools to disk), so it can't fill or balloon the monitored host.

11. Deployment & Operations

The stack runs via Docker Compose: web (Django + Gunicorn, code bind-mounted), db (PostgreSQL 15), redis (Redis 7), and an optional ollama. A production overlay (docker-compose.prod.yml) adds a containerized nginx and certbot.

Install & first run

setup.sh generates a .env with strong secrets, writes the nginx TLS config (one of three SSL modes: Let's Encrypt HTTP-01 auto-renew, upload-your-own, or self-signed; supports non-443 forwarded ports), brings the stack up, then a one-time web wizard at /setup creates the initial admin (the only admin path).

Updates

update.sh performs test -> migrate -> restart -> health-check with automatic rollback. Day-to-day: git pull then recreate the web container; on boot the container runs migrate and collectstatic before Gunicorn starts.

Runtime model

The web container runs migrations and collectstatic, launches the background scheduler, then execs Gunicorn. Status, charts, and counts are computed live; Redis caches the app heartbeat and assorted summaries.

12. Configuration & Data Retention

- > Secrets and runtime flags live in .env (BEHIND_PROXY, LLM_ENABLED, DB creds, SMTP, ...).
- > Per-server thresholds, monitored disks/services, and anomaly sensitivity are set in MonitoringConfig.
- > Raw metrics are aggregated to hourly/daily rollups; old rows are pruned daily to the configured retention window.
- > Tunables include the offline threshold (OFFLINE_THRESHOLD_SECONDS, default 180s) and the data-retention period.

13. Current State & Recent Enhancements

- > Alert taxonomy (5 categories + per-alert severity) and role-based routing to email/Slack.
- > Anomalies decoupled from alerts: a global dashboard bell with view + clear-all; they no longer appear on the alerts page or change server status.
- > Anomaly explanations now name the heaviest process at the exact sample.
- > Service detection: listening-port + privilege-free banner grab to identify nginx/Apache/LiteSpeed/Exim/Dovecot/etc.; ephemeral/bind-mount disks excluded from disk alerts.
- > RBAC UI gating: the support Operator can view everything but mutating controls (edit, regenerate token, monitoring toggles, suspend, delete) are shown disabled.
- > Status no longer flaps offline on a single transient missed push (180s tolerant threshold).
- > Trend charts: x-axis spans the selected window (24h/7d/30d) with blanks where no data.
- > First-run setup wizard, setup.sh (3 SSL modes), and update.sh with auto-rollback.
- > Comprehensive automated test suite (300+ server tests) covering ingest, RBAC, alerts, detection, retention, and the setup wizard.

Appendix A: Ingestion & Key API Endpoints

Endpoint	Auth	Purpose
/api/agent/metrics/	Token	Receive a system-metrics sample
/api/agent/services/	Token	Sync detected services
/api/agent/containers/	Token	Sync detected containers
/api/agent/ssh-auth/	Token	Ingest SSH auth events
/api/agent/heartbeat/	Token	Liveness ping
/api/anomalies/notifications/	Session	Global unresolved-anomaly feed (bell)
/api/anomalies/clear-all/	Session	Resolve all anomalies
/api/dashboard/summary-stats/	Session	Dashboard counts (alerts only)
/health/ , /ready/	Public	Liveness / readiness probes
/setup/	Gated	First-run admin creation wizard

Appendix B: Scheduled Jobs (metrics_scheduler.py)

Job	Cadence	Purpose
detect_anomalies	5 min	Baseline anomaly detection
detect_memory_leaks	hourly	System/process/IPC leak detection
detect_security_events	1 min	SSH brute-force / auth analysis
run_synthetic_checks	30 s	Uptime / TCP / HTTP checks
check_server_connectivity	1 min	Down/recovered detection
aggregate_metrics	daily	Roll raw metrics into hourly/daily
prune_old_data	daily	Enforce the retention window
track_app_heartbeat	loop	App-liveness for the offline grace period

Appendix C: Roles & Capabilities

Role	Capabilities
Admin	All capabilities, including user & role administration and impersonation.
CEO	Everything operational and business; NO user/role admin, NO impersonation. Defaults to the Executive view.
Operator (support)	View Operations only - read-only. Mutating controls are visibly disabled and server-enforced.
(custom roles)	Any subset of capabilities: view operations/executive, manage monitoring/alerts/security/business, users, roles, impersonate.